

Model choice, context window, and cost awareness from the API

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 7 of 12 · English notes

After Hello Claude, Vijay listed models via the SDK (`client.models.list`), distinguished **alias** ids from **dated snapshots**, then walked pricing: input tokens, output tokens, and prompt-cache write/hit. He also spent a long stretch on **temperature** (and top-k / top-p where still exposed). The Foundation exam wants you to know what you are paying for and why pinning a snapshot matters.

Listing models and reading ids

The SDK can list models available to your key, with an id and a display name. What you put in `model :` is the **id**. Families in the August 2026 room were Opus / Sonnet / Haiku style names (including “4.5” dated strings such as a November 2025 snapshot in the spoken example). **Always re-check the live model page before the exam** — ids and windows move.

Alias versus dated snapshot

An alias (class: “generic” / latest in a line such as Haiku 4.5) tracks the newest compatible snapshot. A full id with a date pins behaviour. Vijay’s rule: if you evaluated prompts or tuned behaviour against one snapshot, **do not silently ride the alias** — a new snapshot can “break” an application overnight. Rajesh asked about expiry / SLA: enterprise catalogues publish supported models; migrating off a retired id is a normal software migration (you can automate an agent to bump ids; it is not magic auto-rewrite).

ALIAS CONVENIENCE VERSUS PRODUCTION

Alias = less ops toil, more surprise. Dated id = reproducibility for evals. Exams like that contrast.

Three “windows” in class language

Idea	What Vijay said in class	How to think about it
Input context	On the order of 272K or 1M tokens depending on model; 1M ids often contain a 1M marker	How much system + messages + files you can send
Output / <code>max_tokens</code>	Cap on generated tokens; demos used 1024; hitting the cap truncates	Not the same as the input window
Prompt cache	Separate capacity and price to store a prefix on the server for minutes/hours	Module 09; do not confuse with Chat Pro usage meters

Classroom 272K / 1M figures were teaching numbers. Real models differ (some 200K, some larger). Scoring source: [models](#).

max_tokens and stop_reason

You cannot know the exact output length in advance. Choose max_tokens from the job (short classification vs long report). If the model still has more to say, generation **stops abruptly** and stop details / stop_reason reflect the token limit. Input size is controlled by what you send; output size is controlled by max_tokens (and stop sequences). Vijay’s “base 1024” is a demo default, not a platform minimum — read current API defaults.

Pricing mental model (API, not Claude Pro)

API bills per million tokens, with **different rates** for input versus output (output was described as much more expensive, e.g. a slide pairing ~\$10 input vs ~\$50 output per million on some Opus-class row — *illustrative, not to memorise*). Cache write for ~5 minutes versus ~1 hour had higher write prices; cache hits were described as on the order of \$1 per million on that slide. Chat with a 1M-token PDF on every turn re-pays input; caching the document prefix is the optimisation. **Claude Pro at ~\$20/month with 5-hour refresh is a different product** — do not apply API token maths to that plan.

- Input, output, cache write, and cache hit are separate meters.
- Session in Chat ≠ API key lifetime. Billing is usage against the key, not “one chat window.”
- Third-party tokenizer websites are not Anthropic; Vijay used one only to show that tokenisation is model-specific.

Temperature, top-k, top-p

Temperature in class: range **0 to 1** (he misspoke then corrected). **0** = more deterministic / “specific”; **1** (SDK default he quoted as 1.0) = more diverse / creative. Movie-idea demo: temperature 0 stayed in a similar orbit; 1.0 jumped genres. Use low temperature for extraction and lookup-in-context; higher for brainstorming. Higher temperature is *creativity*, not automatically “hallucination,” though sloppy prompts plus high temperature wander more.

He sketched transformers: embeddings turn text into vectors; attention uses key/query/value; stacked blocks propose next-token distributions. **Top-k** keeps only k candidates; **top-p** (nucleus) further filters by cumulative probability. Important caveat from the room: **newer Claude models often hide or ignore top-k/top-p** and may only expose temperature. Haiku-class models in that week might still accept them. Confirm on the model’s API page.

What belongs in every request

Stateless REST: each call carries model id, messages, system (if used), temperature, max_tokens, etc. You cannot “set system once” on the server. Cache is the feature that makes a large prefix cheaper to reuse — not a hidden session.

PRICES AND WINDOWS GO STALE

Any dollar figure or 272K window from 17 August 2026 is a snapshot. Exam and production: current pricing and model cards on docs.claude.com.

Official documentation

- **Models**

<https://docs.claude.com/en/docs/about-claude/models>

Ids, aliases, context windows, snapshot policy.

- **Messages API**

<https://docs.claude.com/en/api/messages>

model, max_tokens, temperature, stop_reason, usage.

- **Prompt caching**

<https://docs.claude.com/en/docs/build-with-claude/prompt-caching>

Cache TTLs and token accounting (full treatment in M09-C04).

- **Rate limits**

<https://docs.claude.com/en/api/rate-limits>

Throughput caps separate from token price.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.